GPU is well known for accelerating parallelizable operations such as matrix multiplication. We consider how it can process multiple jobs simultaneously from high level and give a few specific examples.

We first start with physical hardware involved in GPU computations:

SSD Storage: (Solid-state Drive) is where dataset and model weights live permanently.

DDR RAM: (Double Data Rate) is system memory dedicated to the CPU. CPU reads the data from SSD and loads it into RAM.

PCI Bus: (Peripheral Component Interconnect) passes the data from RAM to VRAM

VRAM: (video random access memory) is a specialized type of memory dedicated to GPU. HBM is one type of VRAM.

SRAM: (Static Random Access Memory) is known as GPU cache. The GPU pulls small tiles of data from VRAM into it for performing operations. (SRAM is usually very small, e.g. 228 KB one SRAM)

Registers: register is the fastest and smallest memory unit in a GPU. One register is dedicated to one thread (while one thread could have many registers). There are different types of registers, such as floating registers and accumulator register.

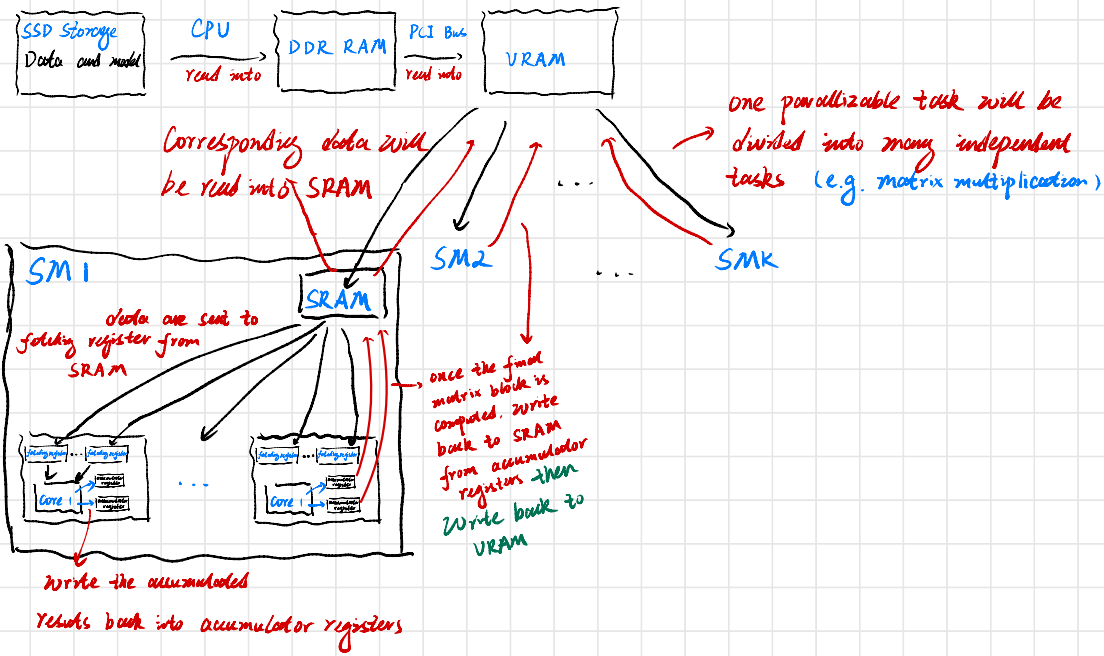
Thread: a sequence of instructions to complete one task.
Eg. to compute 1024×1024 matrix, a tiny 8×8 sub-block of the final matrix is assigned to a worker. The sequence of instructions to finish this task is a thread.

Load data from SRAM to register $\rightsquigarrow$ multiple the numbers $\rightsquigarrow$ get the results to register $\rightsquigarrow$

Computer core: a single unit for performing mathematical operations.

Streaming Multiprocessor: one GPU has many SMs (e.g. NVIDIA H100 has 132 SMs)
Each SM has many Computer cores. Its own SRAM and registers.

High level overview of how GPU works



Specific example: matrix multiplication

Say we have matrix $A \in \mathbb{R}^{1024 \times 1024}$ and $B \in \mathbb{R}^{1024 \times 1024}$ and we want to compute $AB = C$ in GPU.

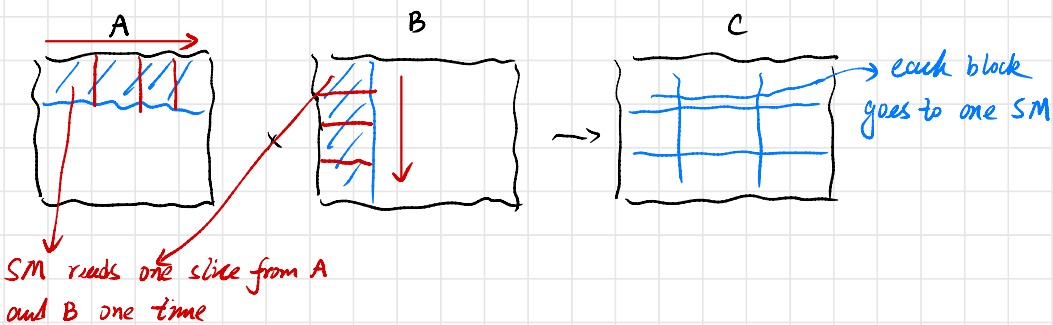
Step 1: output division

The GPU's scheduler looks at the 1024×1024 matrix C and divide it into a grid of smaller output's blocks (e.g. 128×128 blocks). Each block will be assigned to one SM.



Step 2: Loading tiles (from HBM to SRAM)

In order to compute the 128×128 block, the SM needs corresponding 128 rows from A and 128 columns from B. However, SRAM might not fit all data. Instead, the SM load a thin slice from HBM, for example a 128×32 from A and 32×128 from B.



Step 3: Register Allocation (from SRAM to register)

One thread will get assigned to compute 8×8 small blocks.
The thread will reserve multiple accumulator and private registers.
(fetching registers)

Step 4: Computation

The thread reads data from feeding registers and adds the results directly into the accumulator registers.

Step 5: Sliding the tile

Once the thread exhaust the data in the current SRAM slice. The SM slides the window. Feeding the next slides from A and B.
Crucially, the accumulator registers hold the running sums until all slices are used.

Step 6: Write back (accumulator registers → SRAM → HBM)

Once the sliding windows reach the end, the accumulator registers hold the final results. They will be written back to SRAM and HBM

Given what we have discussed so far, some ml observations when using GPUs

1. Dealing with one batch (e.g. 32) often takes almost the same time as a single sample.

To process a single or a batch of samples, the GPU reads the model and the data into SRAM (where model is often much larger than the data in one batch) and then perform computations. For both, the model will be loaded into SRAM once. The loading takes most of the time and computation is normally very fast.

2. What is memory bound (e.g. LLM inference)

It suggests that GPU compute cores spends more time waiting for data to arrive from HBM to SRAM than they spend actually performing math